

VisionBridge — Offline OCR tonight: your action plan

Goal: the existing **Smart Reading (“Read Text”)** feature reads printed text with a **fine-tuned OCR model on the phone** — no internet, no Gemini, no Supabase — while every other VisionBridge feature keeps working exactly as before.
Antigravity writes all the code, builds the dataset, trains, converts, integrates, builds and installs the app. Your job is only the parts a computer cannot do.

TONIGHT’S ACTION PLAN — the only things you do yourself

#	You do this yourself	When / how long
1	Connect your phone to the laptop by USB and switch on USB debugging.	Start · 10 min
2	Open the project in Antigravity, paste the prompt (Section “Prompt” of this PDF) and send it.	Start · 2 min
3	Approve the prompts Antigravity shows (installers, commands, phone dialogs).	Whenever they pop up
4	Take ~30 photos with your phone camera when Antigravity asks for them.	After ~20 min · 15 min
5	Correct ~50 pre-filled labels in the review page Antigravity opens in your browser.	After the photos · 10 min
6	Test on the phone in airplane mode and tell Antigravity what you heard.	Late · 15 min

That is the whole list — about **45 minutes of your attention**, spread across the evening. You do **not** install Python, download models, write scripts, label a full dataset, run training commands, convert models, write Kotlin, or install the APK by hand: Antigravity does all of that.

Have these ready before you start

- Your phone + a working USB cable (data cable, not charge-only), and the phone unlocked.
- The laptop on mains power, set to **never sleep** tonight (training runs in the background).
- A printed page, a product box or a medicine box, and one small-print label to test with.
- The VisionBridge project already open in Antigravity, and your VisionBridge master context document at hand.

Rough timeline

Time	What is happening	You?
0:00–0:15	Phone connected · prompt pasted · Antigravity inspects the repo and makes a safety commit	Yes (steps 1–2)
0:15–0:40	Antigravity installs tools, downloads models, builds synthetic data; you take photos	Yes (step 4)
0:40–0:55	Antigravity crops + auto-labels; you confirm ~50 labels in the browser	Yes (step 5)
0:55–2:30	Training runs in the background while Antigravity writes and builds the Android side	No — free time
2:30–3:00	Model swapped in, APK installed, you test offline, Antigravity fixes and reports	Yes (step 6)

If the laptop has no NVIDIA GPU, training is slower (1–2 h) but still runs in the background — Antigravity shrinks the recipe automatically and keeps working on the Android side meanwhile.

How to talk to Antigravity tonight

- If it asks you to write code, create folders, run training commands or convert a model: reply “Please do that yourself – I only do the physical steps.”
- If anything errors: paste the **whole** error and say “Diagnose and fix this yourself, then continue.”
- Every message it sends should end with **NEXT ACTION FOR YOU**. If it doesn’t, ask: “What exactly do I do next?”

THE 10 STEPS, IN ORDER

STEP 1 — Prepare computer and phone		10 min · mostly you
YOU DO	1. Plug the phone into the laptop with a USB data cable. 2. On the phone: Settings → About phone → tap “Build number” 7 times → back → System → Developer options → USB debugging: ON. 3. When the phone asks “Allow USB debugging?” , tick “Always allow” and tap Allow . 4. Set the laptop to never sleep tonight (Settings → System → Power & battery → Screen and sleep → Never) and keep it on mains power. 5. Open the visionbridge project folder in Antigravity and allow it to run terminal commands and edit files.	
ANTIGRAVITY DOES	Checks git, makes a safety commit and a new branch feature/offline-ocr, checks Java, Python, adb and whether the laptop has an NVIDIA GPU, then installs everything it needs (Python, PaddleOCR, ONNX tools). You only approve the Windows installer prompts.	
YOU CHECK	It reports your phone under adb devices and “checkpoint commit created”. If it says <i>no device found</i> : re-plug the cable, unlock the phone, tap Allow again.	

STEP 2 — Hand Antigravity the prompt (it then inspects everything)		2 min · you
YOU DO	1. Open docs\offline-ocr\VisionBridge_Offline_OCR_Antigravity_Prompt_v2.txt (saved inside your project, same text as the last section of this PDF). 2. Copy all of it into Antigravity's chat, attach or paste your VisionBridge master context document in the same conversation, and send. Say to Antigravity: "Work autonomously. Only ask me for things I physically have to do."	
ANTIGRAVITY DOES	Inspects the real code before touching anything: Smart Reading screen and ViewModel, the camera/capture path, the current OCR call, TTS, accessibility, Gradle files, AndroidManifest, dependencies, existing model/inference code and everything MedSafe and the other features share. Then posts a ~10-line map and a minimal plan and keeps working without waiting for you .	
YOU CHECK	The map names real files: SmartReadingScreen.kt → SmartReadingViewModel.kt → SmartReadingRepository.kt → SmartReadingApi.kt → reading-extract, and says MedSafe / Live / Volunteer Help stay untouched. If it invents file names, reply: "Re-inspect the repository; do not assume file names."	
STEP 3 — Choose the model		0 min · not your decision
YOU DO	Nothing. Let Antigravity choose after inspection. The prompt already names the recommended pair; Antigravity either confirms it or replaces it and tells you why in one line.	
ANTIGRAVITY DOES	Selects the lightweight model, decides the fine-tuning approach and the mobile format (ONNX + ONNX Runtime on Android unless it finds a real blocker), weighs size against speed, downloads the models and records the decision. Recommended starting pair: PP-OCRv5_mobile_det (4.8 MB, finds text boxes, used as-is) + en_PP-OCRv5_mobile_rec (7.5 MB English recognizer, this is the one that gets fine-tuned).	
YOU CHECK	One line in chat naming both models, their sizes, the format, and that they are bundled in the APK with no network access. Nothing else to verify.	
STEP 4 — Give Antigravity your photos		15 min · your main input
YOU DO	1. When Antigravity asks, take about 30 photos with your phone's normal camera app: printed page · receipt · sign · room or bus number · product packaging · medicine box · small print · curved bottle label · dim room · slight hand shake · angled shot · glossy glare · a screen. 2. Keep the phone plugged in and reply: "Photos are on the phone — pull them yourself." 3. Only if it reports that pulling failed: copy the photos into the folder it names (ocr-training\data\real_photos\). 4. Do not photograph private documents (ID cards, bank papers).	
ANTIGRAVITY DOES	Pulls the photos over adb; generates 6,000–10,000 synthetic training images in VisionBridge conditions (blur, glare, low light, perspective, JPEG noise); detects and crops every text line; auto-labels the training crops; splits training and validation by photo ; writes the label files, the augmentation pipeline and the training configuration. You never write an annotation file.	
YOU CHECK	A count like "31 photos → 214 crops · 165 auto-labelled for training · 49 for you to verify".	
STEP 5 — Confirm about 50 labels		10 min · the only typing
YOU DO	1. Open the link Antigravity gives you — normally http://localhost:8765. 2. For each cropped image: if the pre-filled text matches, click Save & next. If it is wrong, fix the text first. If you cannot read it, click Delete. 3. Type exactly what you see (same capitals and punctuation) and never guess. Close the page when it says finished.	
ANTIGRAVITY DOES	Builds and runs the review page, pre-fills every text with the model's guess, saves each correction instantly, validates the file and keeps this set strictly out of training.	
YOU CHECK	"Validation set ready: N labels confirmed" and the counter reached the end. <i>Why it matters: this is the only honest way to prove the fine-tuned model is actually better than the stock one.</i>	
STEP 6 — Fine-tune the model		0 min of yours · 45–120 min waiting
YOU DO	1. Leave the laptop on, plugged in and awake. Approve any prompt. 2. That is all — do not run training commands yourself.	
ANTIGRAVITY DOES	Measures the stock model's accuracy on your 50 verified crops first, starts training in the background (log: ocr-training\logs\train.log), monitors it, fixes crashes (batch size, workers, memory) by itself, keeps the best checkpoint, re-measures, exports the model, verifies the export and copies the final file into the Android app. Meanwhile it continues with Step 7.	
YOU CHECK	A before/after table, e.g. baseline acc 0.71 → fine-tuned acc 0.83, and the file app\src\main\assets\ocr\en_rec.onnx (~8 MB). If fine-tuning did not improve things, Antigravity must say so and ship the stock model — that is an acceptable, honest outcome.	

STEP 7 — Android integration

0 min · all Antigravity

YOU DO	Nothing. Keep the phone connected and approve prompts. Do not write Kotlin.
ANTIGRAVITY DOES	Adds the ONNX Runtime dependency, bundles the models as app assets, writes the OCR engine (loaded once, runs on a background thread, decodes text and a confidence score), connects it only to Smart Reading, reuses the existing camera capture, TTS, UI and TalkBack behaviour, forbids network/Gemini/Supabase in that path, handles lifecycle and memory, then builds the APK and installs it on your phone itself.
YOU CHECK	“APK built and installed on <your phone>”, plus a sentence stating the OCR path makes no network calls. The Read Text screen should look exactly as before — only what happens after you press capture has changed.

STEP 8 — Test it on the phone

10 min · you

YOU DO	1. Unlock the phone, open VisionBridge and log in as usual. 2. Open Read Text (Smart Reading). 3. Point the camera at a printed page from about 20-30 cm and hold steady. 4. Press capture and listen. 5. Repeat with: a sign or product box, small print, a dim room, and something glossy with glare. 6. Tell Antigravity what happened — if the text was wrong, type what it said and what the page really says.
ANTIGRAVITY DOES	Reads logcat, fixes crashes and inference errors, measures how long each capture takes, tunes settings, rebuilds and reinstalls, and tells you when to re-test.
YOU CHECK	PASS: text is spoken within a few seconds and mostly matches the page. FAIL: app crashes, says nothing, speaks nonsense, or takes more than ~5 seconds → send the symptom to Antigravity and let it fix it.

STEP 9 — Prove it is really offline

5 min · you

YOU DO	1. On the phone turn Airplane mode ON. 2. Check that Wi-Fi is OFF and mobile data is OFF (airplane mode sometimes leaves Wi-Fi on). 3. Open Read Text and capture the same printed page again. 4. Listen, then report: “Offline test: text was read aloud / nothing was read.”
ANTIGRAVITY DOES	Audits the OCR code path for any hidden network dependency (HTTP client, Supabase, Gemini, runtime model download), watches logcat during your capture to show that no request was made, and reports that evidence.
YOU CHECK	Text is still read aloud with all radios off → the OCR genuinely runs on the phone. If it works online but not offline, tell Antigravity — something still calls the server and it must remove it.

STEP 10 — Final verification

10 min · you

YOU DO	1. Turn the network back on and quickly open: login and home, AI Vision, Currency, Object Finder, VisionBridge Live, AI Voice Call, Volunteer Help, MedSafe scan, News or Radio, and the SOS screen (open it only — do not trigger it). 2. Ask Antigravity for its final report and read the two lists it must produce: YOU STILL NEED TO DO and I ALREADY DID FOR YOU. 3. Tick the checklist below.
ANTIGRAVITY DOES	Produces the final report with measured numbers (model size, OCR time, accuracy before/after, device tested), known limitations and any remaining action — and fixes whatever you report as broken.
YOU CHECK	Everything in the checklist below is ticked, and nothing in the report is marked NOT VERIFIED that you care about tonight.

IF SOMETHING GOES WRONG — what to send to Antigravity

What you see	What you send (copy this)
It asks you to write code, create folders or run a training command	“Please do that yourself — I only do the physical steps. Continue autonomously.”
Any error message, anywhere	Paste the whole message, then: “Diagnose and fix this yourself, then continue.”
Training looks stuck	“Check ocr-training/logs/train.log, tell me the last line and whether loss is still improving.”
It is getting late and training is not done	“Stop training, ship the pretrained recognizer through the same Android code, and finish the offline test tonight.”
The app crashes when you capture	“It crashes on capture. Read logcat, fix it, rebuild, reinstall, and tell me when to retest.”
The spoken text is nonsense	“OCR output is garbled. Verify the dictionary order, BGR channel order and image rotation against your Python reference, then fix it.”
Works with internet, fails in airplane mode	“The OCR path still uses the network. Find it, remove it, and prove it from logcat.”
It says “done” without testing on the phone	“Did you run this on the phone? Mark anything unverified as NOT VERIFIED and test it now.”

FINAL CHECKLIST

- ☐ Model is genuinely fine-tuned (or Antigravity clearly states it shipped the pretrained model)
- ☐ Final model file is bundled inside the Android app (assets), not downloaded
- ☐ Smart Reading works with airplane mode ON, Wi-Fi OFF and mobile data OFF
- ☐ No Gemini / Supabase / API call happens when reading text
- ☐ The recognized text is spoken by the existing text-to-speech
- ☐ A capture finishes in a few seconds on your phone
- ☐ Camera and the Read Text screen behave as before
- ☐ Other features still work: Live, Volunteer Help, MedSafe, Vision, Currency, News
- ☐ APK builds and installs
- ☐ Tested on your real phone, not only in the emulator

THE ANTIGRAVITY PROMPT — copy everything below

Same text as docs\offline-ocr\VisionBridge_Offline_OCR_Antigravity_Prompt_v2.txt in your project — copy it from that file so page footers don't end up inside the prompt. Paste your VisionBridge master context into the same conversation.

ROLE

You are the primary implementation agent for adding a fine-tuned, fully offline OCR model to an existing Android project called VisionBridge. You have the complete repository, a terminal, and my approval to run commands. I am a beginner in machine learning and model deployment: do the work yourself, and speak to me in plain language.

OBJECTIVE

Tonight, the existing Smart Reading ("Read Text") feature must read printed text with an on-device, fine-tuned OCR model instead of the cloud: camera capture -> offline OCR -> text -> the existing accessible UI and text-to-speech. No other VisionBridge feature may change.

AUTONOMY RULE (most important)

Do as much of the work yourself as possible. Never ask me to do something you can do through the repository, the terminal, scripts, model tooling or adb. Only ask me when:

- human input is genuinely required (taking real photos, correcting a small set of labels),
- a permission, credential or UAC/install prompt needs my click,
- physical phone testing is required,
- an irreversible or scope-changing decision genuinely needs my choice.

Otherwise proceed on your own and tell me afterwards what you did.

Bad: "You need to create the training folder / add the dependency / convert the model."

Good: "I created ocr-training/, added the ONNX Runtime dependency and exported the model; you only need to put photos in ocr-training/data/real_photos/."

End EVERY message with one line: "NEXT ACTION FOR YOU: <one concrete thing>" or "NEXT ACTION FOR YOU: nothing - I am continuing."

TIME BUDGET AND PRIORITY ORDER

I have one evening. Optimize in this order: a working end-to-end implementation first, then accuracy and speed, then polish. Minimum viable sequence: fine-tuned model -> export -> Android integration -> offline test -> fix -> optimize. Do not build a research pipeline. Do not run experiments that are not needed to ship tonight. Run long jobs (training) in the background and keep working on the Android side while they run.

SCOPE PROTECTION (hard limits)

This task is ONLY: VisionBridge -> Smart Reading -> offline OCR.

Do not redesign, rewrite, refactor or "improve": Gemini features, VisionBridge Live, AI Voice Call, Volunteer Help, WebRTC, SOS, News, Radio, Stories, Games, gamification, authentication, Supabase (database, RLS, Edge Functions), MedSafe, navigation, or any unrelated UI.

MedSafe's pipeline and its 0.70 confidence gate must remain untouched.

Do not modify the Web App Node.js backend (it is not even in this repository).

Do not delete the existing cloud reading path; it stays in the code, just unused by Smart Reading.

Do not add ML Kit, TensorFlow Lite, OpenCV, Tesseract or any NDK/C++ layer.

Do not create a second camera system: reuse the existing CameraX capture in the Smart Reading screen.

WORK ORDER - DO THESE 13 STAGES IN ORDER

1. INSPECT (read-only, ~10 minutes)

Search and read the actual code before changing anything. Do not trust any document, including this prompt, over the repository.

- Project: settings.gradle.kts, both build.gradle.kts, gradle/libs.versions.toml, app/src/main/AndroidManifest.xml, the whole package com.example.visionbridge.
- Trace Smart Reading end to end. Expected (confirm or correct): MainActivity.kt -> ui/screens/SmartReadingScreen.kt (CameraX Preview + ImageCapture, JPEG bytes + rotationDegrees, TextToSpeechManager, TalkBack semantics, ScreenActionRegistry "reading") -> ui/screens/reading/SmartReadingViewModel.kt (ReadingUiState, LOW_CONFIDENCE_THRESHOLD = 0.70, ContextMemoryManager) -> data/SmartReadingRepository.kt (ImageHelper.optimizeAndEncode -> SmartReadingApi) -> api/SmartReadingApi.kt -> Supabase Edge Function reading-extract (Gemini) -> api/ApiResponse.kt ReadingExtraction(extractedText, confidence, message).
- Also read: utils/ImageHelper.kt and every caller of it (MedicationRepository, CurrencyReaderViewModel, ObjectFinderViewModel, SurroundingsViewModel, TransportViewModel), utils/TextToSpeechManager.kt, voice/ScreenActionRegistry.kt, voice/SmartReadingGroundedQa.kt, data/ContextMemoryManager.kt, ai/ConfidenceEvaluator.kt, data/MedicationModels.kt, the MedSafe files, res/values*/strings.xml, app/src/androidTest/java/.../SmartReadingEndToEndTest.kt and app/src/androidTest/assets/reading-sample.jpg.
- Search the repository for any existing OCR/ML/inference code or dependency (ML Kit, TFLite, ONNX, Tesseract, "ocr", "TextRecognition", "Interpreter", assets handling) and reuse whatever exists instead of duplicating it.
- Check the machine: git status/branch, java -version, python --version, adb devices, nvidia-smi (is there an NVIDIA GPU?), free disk space.

2. PLAN (~5 minutes, then keep going)

Post a short map and plan in chat (about 10 lines): Smart Reading entry point, screen, ViewModel, capture source, current OCR call, TTS path, the exact files you will create, the exact files you will modify, the files you will not touch, and whether any other feature shares the Smart Reading OCR path. Then continue immediately - do not wait for my approval. I will interrupt you if something looks wrong.

Before your first edit, create a safety checkpoint yourself:

```
git add -A
git commit -m "Checkpoint before offline OCR"
git checkout -b feature/offline-ocr
```

(If git reports that .git/index.lock exists, delete that file and retry.)

3. PREPARE (environment + folders + scripts; you do all of it)

- Create a top-level folder ocr-training/ (NOT inside app/) with subfolders data/, config/, scripts/, models/, logs/, reports/, and a .gitignore that excludes data/, models/*, pdparams, logs/ and any zip.
- Set up a local Python environment there: `python -m venv ocr-training\venv` (install Python 3.12 with winget first if python is missing; ask me to approve the installer prompt). Use `ocr-training\venv\Scripts\python.exe` for everything after that.
- Install, choosing the build from the nvidia-smi result:
GPU present: `pip install paddlepaddle-gpu==3.3.0 -i https://www.paddlepaddle.org.cn/packages/stable/cu126/`
CPU only: `pip install paddlepaddle==3.3.0 -i https://www.paddlepaddle.org.cn/packages/stable/cpu/`
then: `git clone --depth 1 --branch v3.7.0 https://github.com/PaddlePaddle/PaddleOCR.git ocr-training/PaddleOCR` ; `pip install -r ocr-training/PaddleOCR/requirements.txt` ; `pip install paddleocr==3.7.0 onnx onnxruntime huggingface_hub` ; `paddlex --install paddle2onnx`
Verify with: `python -c "import paddle; paddle.utils.run_check()"`. If a pinned version is unavailable, use the nearest stable one and print what you actually installed.
- Download the models (`curl.exe -L` or `huggingface_hub`):
detector ONNX: https://huggingface.co/PaddlePaddle/PP-OCRv5_mobile_det_onnx/resolve/main/inference.onnx (+ `inference.yml`)
recognizer ONNX baseline: https://huggingface.co/PaddlePaddle/en_PP-OCRv5_mobile_rec_onnx/resolve/main/inference.onnx (+ `inference.yml`)
recognizer weights:
https://paddle-model-ecology.bj.bcebos.com/paddlex/official_pretrained_model/en_PP-OCRv5_mobile_rec_pretrained.pdparams
This is the recommended model pair: PP-OCRv5_mobile_det (4.8 MB, language-independent text boxes, used pretrained) + en_PP-OCRv5_mobile_rec (7.5 MB, English, 505-symbol dictionary) which you fine-tune. Keep it unless inspection shows a concrete blocker; if you change it, say why in one line and stay fully offline.
- Copy configs/rec/PP-OCRv5/multi_language/en_PP-OCRv5_mobile_rec.yml to `ocr-training/config/vb_en_rec.yml` and EDIT THE FILE (do not fight PowerShell quoting with -o overrides): `absolute_data_dir` and `label_file_list` paths, `Global.use_gpu` (true/false), `Global.epoch_num`, `Global.save_model_dir=./output/vb_en_rec`, `Global.pretrained_model`, `Optimizer.lr.learning_rate=0.0002`, `Optimizer.lr.warmup_epoch=1`, and on Windows set `Train.loader.num_workers=0` and `Eval.loader.num_workers=0`. Do not change the character dictionary, `max_text_length` (25) or the architecture.

4. DATA (you build everything except the photos themselves)

- Synthetic set, `scripts/make_synthetic.py`: 10,000 crops on GPU, 6,000 if CPU-only. Each crop is one line of text, 1-25 characters, using only characters from `PaddleOCR/ppocr/utils/dict/ppocrv5_en_dict.txt` plus space. Text from templates that match what a low-vision user photographs: signs, room/bus/platform numbers, product and food packaging, printed documents, receipts, menus, prices with the rupee sign, dates such as "EXP 08/2027", units such as "500 mg" and "10 ml", phone numbers, emails, plus common English words. Render with Pillow using 15+ open-license fonts (regular, bold, condensed, serif, mono), dark-on-light and light-on-dark, varied backgrounds. Augment with `OpenCV/NumPy`: Gaussian and motion blur, low light + noise, perspective, rotation up to +/-7 degrees, glare spot, shadow, JPEG quality 30-90, downscale-then-upscale. Fixed random seed. Write `synth_train.txt` lines as `"synth/000001.jpg<TAB>text"`.
- Real photos: ask me ONCE, in a single short message, to take about 30 photos with my phone, listing exactly what to photograph (printed page, receipt, sign, room/bus number, product packaging, medicine box, small print, curved bottle label, dim light, slight shake, angled shot, glossy glare, phone/monitor screen) and saying not to photograph private documents. Then pull them yourself with adb (`adb shell ls -t /sdcard/DCIM/Camera`, adb pull the newest ones) into `ocr-training/data/real_photos/`. Only if adb pull fails, ask me to copy the folder manually. While waiting for the photos, continue with the synthetic set and with stage 9 (Android integration) using the pretrained models.
- Split the PHOTOS (not the crops) about 70/30 so no photo contributes to both training and validation.
- Crop and label automatically, `scripts/prelabel.py`: run pretrained PaddleOCR on every photo (use `use_doc_orientation_classify=False`, `use_doc_unwarping=False`, `use_textline_orientation=False`, `text_recognition_model_name="en_PP-OCRv5_mobile_rec"`, `text_det_limit_type="max"`, `text_det_limit_side_len=960`, `text_det_unclip_ratio=2.0`), save each detected text crop and its predicted text, and keep only crops of 25 characters or fewer.
 - Training photos: keep crops whose recognition score is ≥ 0.95 as pseudo-labels (no human work). Write `real_train.txt`, and `real_train_x10.txt` repeating those lines 10 times so real data is not drowned by synthetic data.
 - Validation photos: take about 50 crops for me to verify.
- Build a tiny local review tool, `scripts/review_labels.py`, using only the Python standard library (`http.server`, no extra dependencies): it serves <http://localhost:8765>, shows ONE crop at a time, large, with the predicted text pre-filled in a text box, plus "Save & next", "Delete (unreadable)" and "Back", saves every correction immediately to `ocr-training/data/real_val.txt`, and shows progress (for example 12/50). Start it yourself and give me the link. Tell me: type exactly what is visible; press Delete if you cannot read it; never guess.
- Validate the dataset automatically: every image exists, every label is 1-25 characters, every character is in the dictionary, no photo appears in both splits. Print the counts.

5. TRAIN (you run it; I do nothing)

- First measure the baseline: evaluate the pretrained recognizer on `real_val.txt` with `ocr-training/PaddleOCR/tools/eval.py` and save `acc` and `norm_edit_dis` to `reports/baseline.json`.
- Then fine-tune with `tools/train.py -c ocr-training/config/vb_en_rec.yml`. Recipe: GPU -> 20 epochs, batch 128 (halve it on out-of-memory); CPU only -> reduce to about 6,000 synthetic crops, 8 epochs, batch 32, and expect 45-120 minutes.
- Launch training as a BACKGROUND process writing to `ocr-training/logs/train.log`, keep working on the Android side, and check the log every few minutes. Tell me once, in one line, that training started and roughly how long it will take.
- Do not tune hyperparameters beyond this unless training clearly fails. One run is enough tonight.

6. VALIDATE (honesty stage)

- Evaluate the fine-tuned best checkpoint (`output/vb_en_rec/best_accuracy.pdparams`) on the same `real_val.txt` and save `reports/finetuned.json`.
- Print one table: baseline vs fine-tuned (`acc`, `norm_edit_dis`, number of validation crops).
- If the fine-tuned model is NOT better, say so in plain words and ship the pretrained recognizer instead. Never describe a model as fine-tuned if it is not, and always state that the real-photo training labels are model-generated pseudo-labels (confidence ≥ 0.95), not human-verified.

7. EXPORT

- python tools/export_model.py -c ocr-training/config/vb_en_rec.yml with Global.pretrained_model set to the best checkpoint and Global.save_inference_dir=./ocr-training/models/vb_en_rec_infer. Expect inference.json, inference.pdiparams, inference.yml. Do not set FLAGS_enable_pir_api=0.
- Convert: `paddlex --paddle2onnx --paddle_model_dir ocr-training/models/vb_en_rec_infer --onnx_model_dir ocr-training/models/vb_en_rec_onnx` (expect inference.onnx). If the PaddleX plugin fails on Windows, try `pip install paddle2onnx` and `paddle2onnx --model_dir ... --model_filename inference.json --params_filename inference.pdiparams --save_file ...`. If conversion still fails, generate a small Colab notebook that does only the export step, give me the link and continue with the pretrained recognizer meanwhile.
- Check the ONNX file: `onnx.checker`; print input and output names and shapes; assert that the number of output classes equals the dictionary size + 2; run the Paddle model and the ONNX model on all validation crops and report the percentage of identical texts (target 98% or higher).

8. OPTIMIZE (only if stages 9-11 are already working)

- Measure first. If OCR is fast enough on my phone (target: under about 2 seconds for a typical page), keep FP32 and skip quantization.
- Otherwise try static QDQ INT8 (`onnxruntime.quantization.preprocess`, then `quantize_static` with about 100 real crops for calibration, `per_channel=True`) and keep INT8 only if accuracy drops by 1 point or less. Never use plain dynamic quantization on convolution layers (ConvInteger is unsupported on the CPU provider).
- Other cheap wins first: detector long side 736 instead of 960, batching the recognition crops, 4 intra-op threads.

9. INTEGRATE (all code is yours to write)

- Put the final files in `app/src/main/assets/ocr/`: `ppocrv5_mobile_det.onnx`, `en_rec.onnx` (fine-tuned if stage 6 approved it, otherwise the pretrained one), `en_rec_dict.json` (the character_dict list read from the SHIPPED model's inference.yml with `yaml.safe_load` - never retyped by hand), `models.json` (model names, sources, fine_tuned true/false, metrics, file sizes, sha256).
- Add `implementation("com.microsoft.onnxruntime:onnxruntime-android:1.30.0")` to `app/build.gradle.kts` in the file's existing style. No manifest or permission changes are needed (CAMERA is already declared).
- Create package `com.example.visionbridge.ocr` with small, readable files:
 - `OfflineOcrEngine.kt` - singleton `getInstance(context)` holding only `applicationContext`; lazily creates one `OrtEnvironment` and the detector + recognizer `OrtSessions` from assets (`SessionOptions: setIntraOpNumThreads(min(4, cores)), OptLevel.ALL_OPT`); suspend `warmUp()`; suspend `read(bitmap): OcrOutput`; all work on `Dispatchers.Default` inside one `Mutex`; releases the sessions in `ComponentCallbacks2.onTrimMemory (level >= TRIM_MEMORY_UI_HIDDEN)` under the same lock and reloads lazily; never closes a session during an inference; loads the models once and never per image.
 - `TextDetector.kt` - if the long side is above 960 px scale down to 960 keeping aspect ratio, then round both sides to the nearest multiple of 32 (minimum 32). Channel order BGR; value = $(x/255 - \text{mean})/\text{std}$ with mean [0.485, 0.456, 0.406] and std [0.229, 0.224, 0.225] applied to B, G, R in that order; NCHW float32 in a direct `FloatBuffer`. From the [1,1,H,W] probability map: threshold 0.3 -> 8-connected regions -> convex hull from each row's left/right extreme pixels -> minimum-area rectangle (rotating calipers) -> drop if short side < 3 -> box score = mean probability inside the rectangle, keep ≥ 0.6 -> expand each side by $\text{area} \times 2.0 / \text{perimeter}$ -> drop if short side < 5 -> scale back to the original image and clip -> at most 1000 boxes -> reading order (top to bottom; boxes within 10 px vertically go left to right).
 - `TextRecognizer.kt` - perspective-crop each box with `Matrix.setPolyToPoly` onto a `Canvas` (bilinear). If crop height $\geq 1.5 \times \text{width}$, rotate 90 degrees counter-clockwise (as PaddleOCR does) and also try the opposite rotation, keeping the higher score. Resize to height 48 keeping aspect ratio; BGR; value = $(x/255 - 0.5)/0.5$. Sort crops by aspect ratio, run batches of 6, batch width = $\max(320, \text{ceil}(48 \times \text{widest ratio}))$, capped at 3200, zero-padded on the right.
 - `CtcDecoder.kt` - labels = ["blank"] + dictionary + [" "]; argmax per time step, skip index 0, collapse repeats; line score = mean of the max probabilities of the kept steps. Throw a clear error if the model's class count is not dictionary size + 2.
 - `OcrModels.kt` - `OcrLine(text, score, box)`, `OcrOutput(lines, fullText, confidence, timingsMs, modelId)`.
 - Bitmap decoding - JPEG bytes to `ARGB_8888` with `inSampleSize` so the long side is at most 2048, then EXIF rotation, else the `CameraX` `rotationDegrees`. Put this in the ocr package or add a `NEW` function to `ImageHelper`: do NOT change `ImageHelper.optimizeAndEncode`, its constants (1024 px, JPEG 75) or its behaviour, because `MedSafe`, `Currency`, `Object Finder`, `Surroundings` and `Transport` all use it.
- Wire it into Smart Reading ONLY, at `data/SmartReadingRepository.kt`: keep `extractText()`'s signature and its `ApiResult<ReadingExtraction>` return type, add a switch (for example `object SmartReadingOcrConfig { const val USE_OFFLINE_OCR = true }`); when true, decode -> `OfflineOcrEngine.read()` -> `ReadingExtraction(text, confidence, message)` and never call `SmartReadingApi`; when false the old cloud path runs unchanged. No automatic cloud fallback. Give the repository an application Context with the smallest possible change (for example `SmartReadingViewModel` becomes `AndroidViewModel(application)`, still created by `viewModel()`), warm the engine up when that `ViewModel` is created, and keep `SmartReadingEndToEndTest` compiling.
- Confidence and safety: drop lines scoring below 0.50; overall confidence = character-weighted mean of all non-empty recognized lines; when nothing readable is left return `extractedText = null` with an actionable message such as "I couldn't find readable text. Hold the phone 20 to 30 cm from the text, add more light, and try again." (add new strings to values, values-hi and values-mr). Never invent or autocomplete text. The existing 0.70 low-confidence banner in `SmartReadingViewModel` stays exactly as it is.
- Accessibility: keep `TalkBack` labels, focus order, `liveRegion` announcements, large high-contrast text, the existing "Reading text. Please wait." speech and all buttons. Success, low confidence, no text and errors must all be spoken, never only shown.
- Logging: tag "VB-OfflineOcr" with model load time, image size, box count, detection/recognition/total milliseconds and overall confidence - text LENGTH only, never the recognized text, never images.
- Tests: a JVM unit test for the CTC decoder and reading order, and an instrumented test `OfflineOcrInstrumentedTest` that runs the engine on `app/src/androidTest/assets/reading-sample.jpg` and asserts the text contains "PARACETAMOL", logging the timings.

10. BUILD

- `./gradlew.bat :app:assembleDebug` and `./gradlew.bat :app:testDebugUnitTest` must pass (report pre-existing failures separately; do not "fix" unrelated code).
- Install it yourself while my phone is connected: `./gradlew.bat :app:installDebug` (or `adb install -r app\build\outputs\apk\debug\app-debug.apk`). Tell me only if a dialog on the phone needs my tap.
- Report the APK size before and after the change.

11. TEST

- Run the unit test and the instrumented test yourself: `./gradlew.bat :app:connectedDebugAndroidTest`
`"-Pandroid.testInstrumentationRunnerArguments.class=com.example.visionbridge.OfflineOcrInstrumentedTest"`
- Read `adb logcat` yourself (`adb logcat -s VB-OfflineOcr VB-ReadingRepo VB-SmartReadingApi AndroidRuntime`) and fix what you find.
- Prove offline operation two ways: (a) audit the code and confirm the ocr package and the offline branch of `SmartReadingRepository` contain no network calls, no Gemini, no Supabase and no runtime downloads; (b) after I put the phone in airplane mode, confirm from `logcat` that a capture produces VB-OfflineOcr timings and no Smart Reading HTTP request.
- Benchmark on the real phone: median detection / recognition / total milliseconds over 10 captures, and peak memory.
- Then ask me, in one short message, to run the physical test: open Smart Reading with airplane mode on, capture a printed page, and tell you what was read aloud.

- Regression check with the network on: Smart Reading, TTS, login, home navigation, voice commands, and that AI Vision / Currency / Object Finder, VisionBridge Live, Volunteer Help and MedSafe still open and work. Show git diff --stat and revert anything unrelated.

12. FIX

When anything fails - an install, a training crash, an export error, a Gradle error, a device crash, garbled OCR output - do this yourself: read the actual error, diagnose it, fix it, retry, and continue. Do not stop at the first failure and do not simply report "there is an error in X". Only involve me if the fix genuinely needs my machine, my click or my phone. If a stage is truly blocked after real attempts, say exactly what you tried, use the fastest legitimate fallback (for example: ship the pretrained recognizer through the same Android code) and keep the rest moving.

Common traps you are expected to solve alone: dataloader hangs on Windows (num_workers must be 0), GPU/CPU wheel mismatch, CUDA out of memory (halve the batch), .git/index.lock left behind, gibberish OCR (dictionary offsets: blank is index 0, space is last; BGR not RGB; wrong normalization), sideways text (EXIF/CameraX rotation applied too late), "ConvInteger not implemented" (use static QDQ or stay FP32), slow inference (reuse sessions, 4 threads, detector long side 736, batch the crops).

13. REPORT

Finish with two clearly separated lists, in this order:

YOU STILL NEED TO DO

Only actions that physically require me, numbered, each one sentence (for example: "Put the phone in airplane mode and capture one printed page in Smart Reading"). If there is nothing, write "Nothing".

I ALREADY DID FOR YOU

Everything you completed automatically, grouped: inspected, created, modified, dependencies added, dataset built, training performed, model exported, integrated, built, tested.

Then a short facts block, with real measured values only. 1. Models shipped (detector and recognizer, source, fine-tuned yes/no) 2. Training actually performed (crop counts, epochs, minutes, baseline vs fine-tuned acc and norm_edit_dis) 3. Model format and sizes, APK size change 4. Median OCR time on the device (detection / recognition / total) 5. Device tested (model, Android version) 6. Offline test result 7. OCR results on easy and difficult images 8. Regression result 9. Known limitations (for example: English only, detector not fine-tuned, upside-down text) 10. Exact next action for anything incomplete.

Never say "complete" because Gradle compiled. Mark anything you did not verify on the real phone as NOT VERIFIED. Never claim a test passed that you did not run, and never claim fine-tuning that did not finish.

CODING AND COMMUNICATION STYLE

- I am a beginner: small files, clear names, comments only where the ML/inference code is non-obvious. No over-engineering, no unnecessary libraries, no duplicate services, no rewriting of working architecture.
- Follow the patterns already in this repository (ApiResponse/ApiError, getInstance(context), "VB-" log tags, existing string resources for en/hi/mr).
- Search before creating. Build and test after each meaningful change. Fix real errors rather than reporting them.
- If a choice is ambiguous, pick the smallest practical option that fits this project, proceed, and explain it in one line afterwards.
- Never invent project details you have not verified in the repository.